Experiment 07

Develop a simple calculator using XMLRPC

Aim

To implement a remote calculator using XML-RPC requests and responses.

Requirements

Python 3.10+; standard-library XML-RPC server and client.

Procedure / algorithm

1. Create a loopback SimpleXMLRPCServer and register the calculate method.

2. Dispatch addition, subtraction, multiplication and division after validating the operands.

3. Connect a ServerProxy client and call the method; XML-RPC serializes arguments/results over HTTP.

4. Handle invalid operations and division by zero as faults; shut down the demo server cleanly.

Complete source / configuration: calculator.py

```
"""XML-RPC calculator server and client with an automatic loopback demo."""
import argparse
import threading
from xmlrpc.client import ServerProxy, Fault
from xmlrpc.server import SimpleXMLRPCServer


def calculate(operation, a, b):
    if isinstance(a, bool) or isinstance(b, bool):
        raise ValueError("Operands must be numeric")
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
        raise ValueError("Operands must be numeric")
    if operation == "add": return a + b
    if operation == "subtract": return a - b
    if operation == "multiply": return a * b
    if operation == "divide":
        if b == 0: raise ValueError("Division by zero")
        return a / b
    raise ValueError("Unknown operation")


def make_server(port):
    server = SimpleXMLRPCServer(("127.0.0.1", port), logRequests=False)
    server.register_function(calculate)
    return server


def demo():
    with make_server(0) as server:
        worker = threading.Thread(target=server.serve_forever)
        worker.start()
        try:
            with ServerProxy(f"http://127.0.0.1:{server.server_address[1]}") as rpc:
                for op in ("add", "subtract", "multiply", "divide"):
                    print(f"{op}(12, 4) = {rpc.calculate(op, 12, 4)}")
                try: rpc.calculate("divide", 12, 0)
                except Fault: print("divide(12, 0) = XML-RPC fault: division by zero")
        finally:
            server.shutdown()
            worker.join()


if __name__ == "__main__":
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("mode", choices=["server", "client", "demo"])
    p.add_argument("--port", type=int, default=8000)
    p.add_argument("--operation", default="add")
    p.add_argument("--a", type=float, default=12)
    p.add_argument("--b", type=float, default=4)
    args = p.parse_args()
    if args.mode == "demo": demo()
    elif args.mode == "server":
        with make_server(args.port) as server:
            print(f"Listening on 127.0.0.1:{args.port}")
            server.serve_forever()
    else:
        with ServerProxy(f"http://127.0.0.1:{args.port}") as rpc:
            try: print(rpc.calculate(args.operation, args.a, args.b))
            except Fault as error: raise SystemExit(error.faultString)
```

Run

From the repository root unless a cd command is shown.

```
python Exercise_07/calculator.py demo
python Exercise_07/calculator.py server
python Exercise_07/calculator.py client --operation multiply --a 12 --b 4
```

Actual output / verification

```
Actual localhost demonstration:
add(12, 4) = 16
subtract(12, 4) = 8
multiply(12, 4) = 48
divide(12, 4) = 3.0
divide(12, 0) = XML-RPC fault: division by zero
```

Expected output - not an execution record

For 12 and 4, the returned values are 16, 8, 48 and 3.0; division by zero returns a fault.

Observations

The demonstration makes actual HTTP XML-RPC calls through localhost; fault responses are distinct from numeric results.

Result

All four operations and the division-by-zero fault were verified through real XML-RPC calls.